GYANMANJARI INNOVATIVE UNIVERSITY

Gyanmanjari Institute of Technology – Faculty of Engineering & Technology

Cloud-Based Software Engineering

Deploying a Flask Web Application on AWS Elastic Beanstalk

Student Name	_____	Enrollment No.	_____
Subject	Advance Software Engineering (METCE11501)	Semester	1 (M.Tech)
Activity	ALA 3	Marks	10

1. Aim

To deploy a simple application on a cloud platform (**AWS**), design its architecture and demonstrate the live deployment. The report includes the deployment steps, the architecture, screenshots, and a brief note on scalability and reliability.

2. Introduction

Cloud-based software engineering builds and runs software on on-demand infrastructure instead of owned servers. AWS **Elastic Beanstalk** is a Platform-as-a-Service (PaaS): the developer uploads the code, and the service automatically provisions EC2 instances, a load balancer, auto scaling and monitoring.

Item	Choice
Cloud platform	Amazon Web Services (AWS), region ap-south-1 (Mumbai)
Service model	PaaS – AWS Elastic Beanstalk (Python 3.11 platform)
Application	Campus Notice Board – small Flask web app with /health endpoint
Tools	Python 3.11, Flask, Gunicorn, AWS CLI, EB CLI, Git (optional)

3. Application Description

Campus Notice Board shows a few notices and the name of the server (instance) that handled the request. Showing the instance name makes load balancing visible when more than one instance is running. A `/health` endpoint is used by the load balancer to check instance health.

4. Cloud Architecture

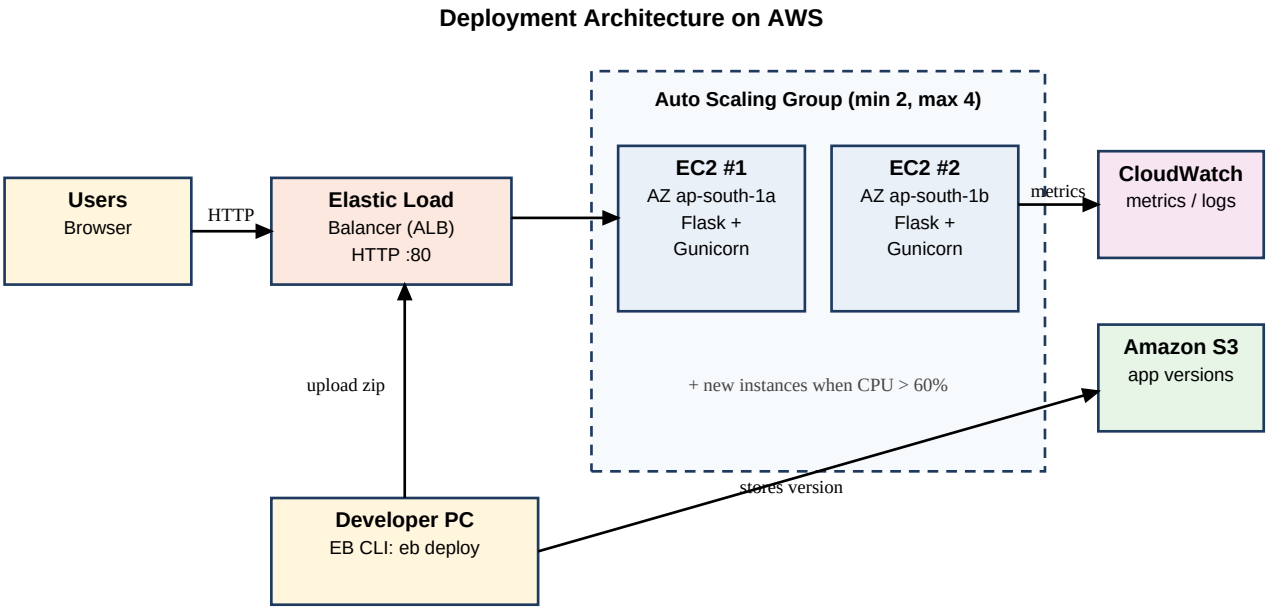


Figure 1: Architecture of the cloud deployment (ALB, Auto Scaling Group across two Availability Zones, S3, CloudWatch).

Component description

Component	Purpose
Application Load Balancer	Single public entry point; distributes requests and removes unhealthy instances using /health.
EC2 instances (t3.micro)	Run the Flask app with Gunicorn. Placed in two Availability Zones.
Auto Scaling Group	Keeps at least 2 instances running; adds instances (up to 4) when CPU is high.
Amazon S3	Stores each uploaded application version (created by Elastic Beanstalk).
CloudWatch	Collects health, CPU and request metrics and application logs.
IAM roles	Give Elastic Beanstalk and the instances only the permissions they need.

5. Application Code

Project folder structure:

```
notice-board/
|-- application.py            # Elastic Beanstalk looks for the object named "application"
|-- requirements.txt
`-- .ebextensions/
    `-- scaling.config        # auto scaling + health check settings
```

5.1 application.py

```
import socket
from datetime import datetime
from flask import Flask, jsonify

application = Flask(__name__)

NOTICES = [
    "Mid-semester exam timetable published",
    "Library open till 8 PM during exams",
    "Submit Active Learning Activity reports on the GMIU Web Portal",
]


@application.route("/")
def home():
    items = "".join(f"<li>{n}</li>" for n in NOTICES)
```

```
    return f"""
    <html><head><title>Campus Notice Board</title></head>
    <body style="font-family:Arial;margin:40px">
      <h1>Campus Notice Board</h1>
      <ul>{items}</ul>
      <hr>
      <p>Served by instance: <b>{socket.gethostname()}</b></p>
      <p>Time (server): {datetime.utcnow():%Y-%m-%d %H:%M:%S} UTC</p>
    </body></html>
    """


@application.route("/health")
def health():
    return jsonify(status="ok")


if __name__ == "__main__":
    application.run(host="0.0.0.0", port=8000)     # local testing only
```

5.2 requirements.txt

```
flask==3.0.3
gunicorn==22.0.0
```

5.3 .ebextensions/scaling.config

```
option_settings:
  aws:autoscaling:asg:
    MinSize: 2
    MaxSize: 4
  aws:autoscaling:trigger:
    MeasureName: CPUUtilization
    Statistic: Average
    Unit: Percent
    UpperThreshold: 60
    LowerThreshold: 20
  aws:elasticbeanstalk:environment:process:default:
    HealthCheckPath: /health
```

6. Deployment Steps

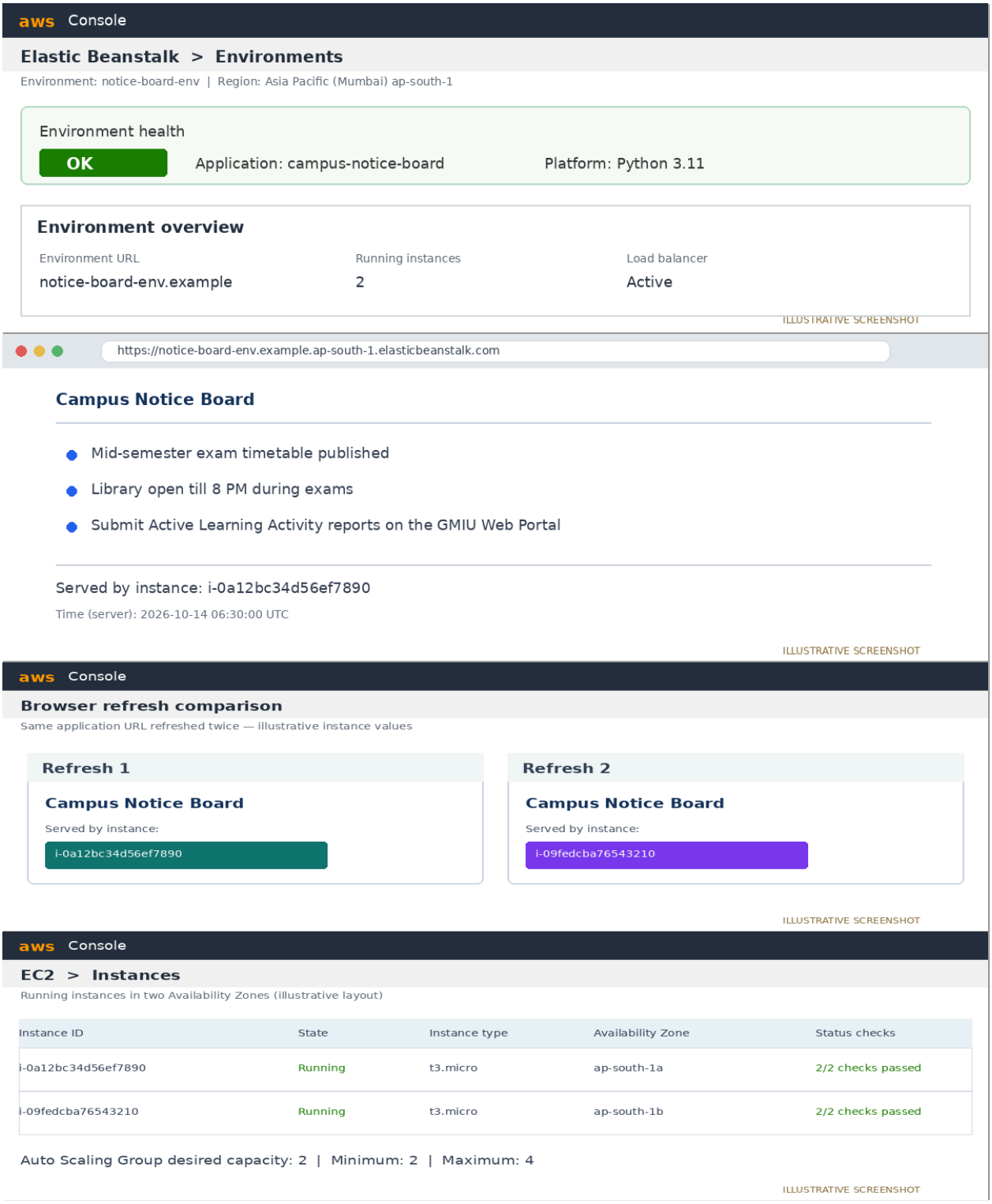
Step	Action	Command / Console action
1	Create an AWS account and an IAM user with programmatic access (avoid using the root account).	AWS Console → IAM → Users → Create user
2	Install and configure the AWS CLI.	`aws configure` (access key, secret, region ap-south-1)
3	Install the Elastic Beanstalk CLI.	`pip install awsebcli`
4	Test the app locally.	`pip install -r requirements.txt`; `python application.py`; open http://localhost:8000
5	Initialise the Elastic Beanstalk application.	`eb init -p python-3.11 campus-notice-board --region ap-south-1`
6	Create the environment (EC2, ALB, Auto Scaling, security groups are created automatically).	`eb create notice-board-env --elb-type application`
7	Check health and open the live site.	`eb status`; `eb open`
8	Verify the health endpoint and load balancing.	Refresh the page repeatedly and watch the instance name change.
9	Update the app (e.g., add a notice) and redeploy with zero downtime.	`eb deploy`
10	Clean up to avoid charges after the demonstration.	`eb terminate notice-board-env`

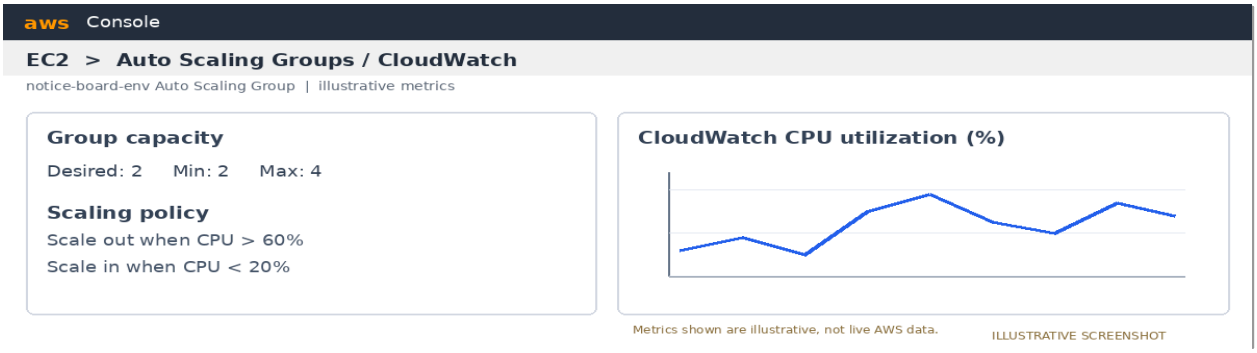
```
# summary of the commands used
```

```
pip install awsebcli
eb init -p python-3.11 campus-notice-board --region ap-south-1
eb create notice-board-env --elb-type application
eb status
eb open
eb deploy                      # after changing the code
eb terminate notice-board-env  # remove all resources when finished
```

Note: Deploy only after checking the AWS Free Tier/cost limits. Running two instances and a load balancer can incur charges, so terminate the environment after taking the screenshots.

7. Live Deployment Demonstration





Live URL: http://_____.ap-south-1.elasticbeanstalk.com

8. Scalability and Reliability

Scalability

- **Horizontal scaling:** the Auto Scaling Group adds EC2 instances (up to 4) when average CPU stays above 60% and removes them below 20%.
- **Stateless design:** the application keeps no data on the server, so any instance can serve any request.
- **Load balancing:** the ALB spreads traffic evenly across all healthy instances.
- **Vertical scaling:** instance type can be changed (for example, t3.micro to t3.small) when more power per instance is required.

Reliability

- **Multi-AZ deployment:** instances in two Availability Zones keep the site running if one data centre fails.
- **Health checks:** the ALB calls /health; failing instances are taken out of rotation and replaced automatically.
- **Rolling deployments:** `eb deploy` updates instances in batches so the site stays available.
- **Monitoring:** CloudWatch metrics and logs allow quick detection of problems.
- **Version history:** previous application versions in S3 allow quick rollback.

Limitations / possible improvements

- Add HTTPS with AWS Certificate Manager and a custom domain via Route 53.
- Store dynamic data in a managed database (Amazon RDS/DynamoDB) instead of a hard-coded list.
- Automate deployment with a CI/CD pipeline (GitHub Actions or AWS CodePipeline).

9. Conclusion

A Flask application was packaged and deployed to AWS Elastic Beanstalk, which created a load-balanced, auto-scaling environment across two Availability Zones. The activity showed how cloud platforms remove infrastructure effort and provide scalability and reliability through load balancing, health checks and automatic recovery.

References

- AWS Elastic Beanstalk Developer Guide – Python platform and EB CLI, docs.aws.amazon.com.
- Flask and Gunicorn documentation.

- Ian Sommerville, *Software Engineering*, 9th ed., Pearson Education, 2010 (Cloud-based software engineering).